Казахский агротехнический исследовательский университет
имени С. Сейфуллина

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

Разработка REST API для управления студентами

Группа: КИ СОПР 05-057-25-22

Выполнил(а): Елдар Б.

Проверил(а): _____

2026

1 Цель и постановка задачи

Разработано серверное приложение student-api на Node.js и Express для управления сведениями о студентах. Реализованы шесть REST-маршрутов, журналирование запросов и два варианта постоянного хранения данных: PostgreSQL и MongoDB. Оба варианта прошли автоматические и сетевые проверки.

Цель работы - закрепить навыки создания серверного приложения, проектирования маршрутов REST API, выполнения CRUD-операций и взаимодействия с базой данных.

В рамках работы выполнены задания: создание проекта; реализация маршрутов; разработка middleware; подключение PostgreSQL; работа с MongoDB; сравнение двух СУБД.

Архитектура приложения

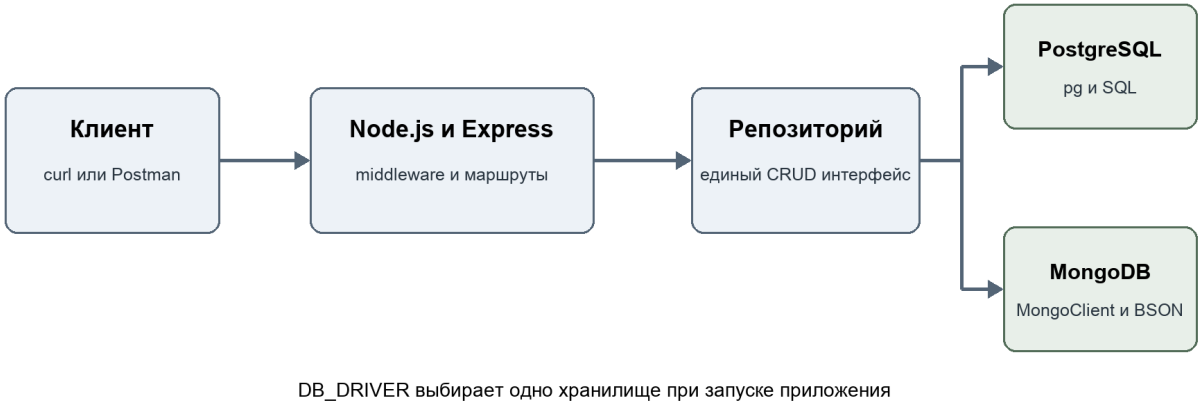


Рисунок 1 - Обработка запроса и выбор хранилища данных

Клиент отправляет HTTP-запрос. Express применяет middleware и выбирает маршрут. Обработчик проверяет данные и вызывает репозиторий. Репозиторий выполняет SQL-запрос либо операцию MongoDB, после чего сервер возвращает JSON и HTTP-статус.

Переменная DB_DRIVER выбирает одно хранилище при запуске. Переключение между СУБД не переносит данные: таблица PostgreSQL и коллекция MongoDB независимы.

2 Создание и организация проекта

Для проекта установлены express, pg, mongodb и dotenv. Файл package.json содержит команды запуска, подготовки данных и проверки. В проекте используются ES-модули: import и export.

```
mkdir student-api
cd student-api
npm init -y
npm install express@5 pg mongodb dotenv
```

Основные файлы

Файл или каталог	Назначение
app.js	Подключение к БД и запуск сервера на порту 3000
src/app.js	Маршруты, middleware и обработка ошибок
src/validation.js	Проверка полей студента и HTTP-ошибки
src/database.js	Выбор PostgreSQL либо MongoDB
src/repositories/	CRUD через pg и официальный драйвер mongodb
sql/schema.sql	Создание таблицы PostgreSQL
scripts/	Инициализация, примеры данных и проверки
test/	Проверки HTTP-контракта и ошибок
evidence/	Подтверждающие журналы выполнения

Запуск сервера

```
npm install
npm run db:init
npm run db:seed
npm start
```

Адрес API - http://127.0.0.1:3000/api/students. Объект Express создаётся в createApp(), а app.listen() вызывается в корневом app.js. Соединение с БД устанавливается до запуска HTTP-сервера.

3 Реализация маршрутов REST API

Метод	Маршрут	Операция	Успех
GET	/api/students	Список студентов	200
GET	/api/students/:id	Один студент	200
POST	/api/students	Создание студента	201
PUT	/api/students/:id	Полное обновление	200
PATCH	/api/students/:id	Частичное обновление	200
DELETE	/api/students/:id	Удаление	204

POST возвращает созданного студента и заголовок Location. PUT требует все поля name, email и group_name. PATCH принимает хотя бы одно из этих полей и сохраняет остальные. DELETE возвращает ответ без тела.

Обработка ошибок

Статус	Причина
400	Некорректный id, JSON или данные студента
404	Студент либо маршрут не найден
409	Email уже используется другим студентом
413	JSON превышает установленный предел 10 КБ
415	Неподдерживаемый Content-Type или кодировка
500	Внутренняя ошибка приложения или запросов к БД

Ошибки возвращаются в едином JSON-формате. Для отсутствующей записи используется корректный по формату id: неверный формат идентификатора возвращает 400, а не 404.

```
{ "error": "Студент не найден." }
```

4 Примеры запросов и проверка данных

Создание студента

```
POST /api/students
Content-Type: application/json

{
  "name": "Алексей Смирнов",
  "email": "alexey@example.com",
  "group_name": "ИС-21"
}
```

Успешный ответ имеет статус 201. В PostgreSQL поле id является числом, в MongoDB - строкой из 24 шестнадцатеричных символов, полученной из ObjectId.

Полное и частичное обновление

```
PUT /api/students/:id
{ "name": "Алексей Смирнов",
  "email": "alexey.new@example.com",
  "group_name": "ИС-22" }

PATCH /api/students/:id
{ "group_name": "ИС-23" }
```

В PUT передаются все три изменяемых поля. В этой реализации PUT обновляет существующую запись и не создаёт новую при отсутствии id. PATCH изменяет только переданные поля.

Правила валидации

name - непустая строка до 100 символов; email - корректная строка до 254 символов; group_name - непустая строка до 50 символов. Краевые пробелы удаляются, email приводится к нижнему регистру. Неизвестные поля, пустые объекты, массивы и объекты вместо строк отклоняются.

Клиент не может назначить id или передать MongoDB-операторы вместо значений. Для запросов POST, PUT и PATCH требуется Content-Type: application/json.

5 Middleware и базовая защита

Middleware журналирования сохраняет время начала обработки, вызывает next() и записывает результат при событии finish объекта ответа. Запись содержит дату и время UTC, HTTP-метод, URL, статус и длительность обработки в миллисекундах.

```
app.use((req, res, next) => {
  const started = Date.now();
  const timestamp = new Date(started).toISOString();
  res.on('finish', () => {
    log(`${timestamp} ${req.method} ` +
      `${JSON.stringify(req.originalUrl)} ` +
      `${res.statusCode} ${Date.now() - started}ms`);
  });
  next();
});
```

Время охватывает обработку на сервере, включая ожидание БД. Событие finish не подтверждает фактическую доставку ответа клиенту, поэтому сетевую задержку до клиента эта величина не измеряет.

Порядок промежуточной обработки

Журналирование → заголовки защиты → express.json() → маршруты → обработка неизвестного маршрута → обработка ошибок.

Реализованные меры защиты

Проверяются поля и идентификаторы, ограничивается размер JSON, SQL выполняется через параметры, а имена колонок PATCH выбираются из фиксированного списка. На уровне БД проверяются обязательные поля и уникальность email.

Отключён заголовок X-Powered-By, установлены nosniff, CSP, запрет встраивания во фрейм и Cache-Control: no-store. Реквизиты вынесены в локальный .env; клиент не получает SQL, пароль или stack trace.

Приложение предназначено для локальной учебной работы. Аутентификация, разграничение доступа и HTTPS не реализованы; при публичном размещении их необходимо добавить.

6 Подключение PostgreSQL

Созданы рабочая база student_api и отдельная база student_api_test. Для подключения используется библиотека pg и пул соединений. Параметры берутся из DATABASE_URL; пользователь на локальном сервере - eldar, порт - 5432.

```
CREATE TABLE IF NOT EXISTS students (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL CHECK (length(trim(name)) > 0),
  email VARCHAR(254) NOT NULL UNIQUE
    CHECK (email = lower(email) AND email ~ '^[^[:space:]@]+@[^[:space:]@]+\.
[^[:space:]@]+$'),
  group_name VARCHAR(50) NOT NULL CHECK (length(trim(group_name)) > 0)
);
```

Идентификатор формируется автоматически. Ограничения NOT NULL, UNIQUE и CHECK защищают обязательные поля, уникальность email и допустимые значения. Данные передаются отдельно от SQL-команды через параметры $1, $2 и далее.

```
const result = await pool.query(
  'SELECT id, name, email, group_name FROM students WHERE id = $1',
  [id]
);
```

Data Output Messages Notifications

	id [PK] integer	name character varying (100)	email character varying (254)	group_name character varying (50)
1	1	Анна Иванова	anna@example.com	ИС-21
2	2	Тимур Ахметов	timur@example.com	ИС-22
3	3	Мария Петрова	maria@example.com	ИС-21

Рисунок 2 - Таблица students в pgAdmin фрагмент настоящего скриншота

Добавлены три записи: Анна Иванова, Тимур Ахметов и Мария Петрова. Фактическая версия сервера - PostgreSQL 16.13; название регистрации «PostgreSQL 18» в pgAdmin является пользовательской меткой.

7 Работа с MongoDB

Установлена MongoDB Community 8.0.34. Соединение выполняется через MongoClient по адресу mongodb://127.0.0.1:27017. В рабочей базе student_api создана коллекция students; база student_api_test используется для проверок.

```
{
  _id: ObjectId("507f1f77bcf86cd799439011"),
  name: "Анна Иванова",
  email: "anna@example.com",
  group_name: "ИС-21"
}
```

Выше приведён пример структуры документа; ObjectId иллюстративный. В ответе API поле _id преобразуется в строковое id, поэтому набор полей ответа совпадает с вариантом PostgreSQL.

Операция	Метод драйвера
Список	find({}).sort({_id: 1}).toArray()
Поиск по id	findOne({_id: id})
Добавление	insertOne(data)
Полное обновление	findOneAndReplace({_id: id}, data, options)
Частичное обновление	findOneAndUpdate({_id: id}, {$set: data}, options)
Удаление	deleteOne({_id: id})

Инициализация задаёт JSON Schema validator и уникальный индекс email. Обязательны name, email и group_name; посторонние поля запрещены. Проверка непосредственно на уровне MongoDB подтвердила отклонение документа без обязательного поля и повторного email.

Для обновлений используются returnDocument: after и includeResultMetadata: false: возвращается итоговый документ либо null, если запись отсутствует. В рабочую коллекцию добавлены три примера студентов.

8 Проверка работоспособности

Для каждого хранилища отдельно выполнена команда проверки. Тесты охватывают CRUD, статусы, обязательные поля, нормализацию, неверный JSON, ограничения тела, Content-Type, повторный email, отсутствие записи и журналирование.

Показатель	PostgreSQL	MongoDB
Успешные тесты	56	56
Ошибки и пропуски	0 и 0	0 и 0
Маршруты по HTTP	6 из 6	6 из 6
Итоговый статус	passed	passed

PostgreSQL | фрагмент сохранённого журнала

```
tests 56
pass 56
fail 0
skipped 0

10:32:17.578 POST "/api/students" 201 10ms
10:32:17.592 GET "/api/students" 200 2ms
10:32:17.594 GET "/api/students/3" 200 2ms
10:32:17.597 PATCH "/api/students/3" 200 2ms
10:32:17.600 PUT "/api/students/3" 200 1ms
10:32:17.602 DELETE "/api/students/3" 204 1ms
10:32:17.603 GET "/api/students/3" 404 1ms
HTTP CRUD: PASS
```

Рисунок 3 - Фрагмент сохранённого журнала PostgreSQL

MongoDB | фрагмент сохранённого журнала

```
tests 56
pass 56
fail 0
skipped 0

10:50:48.554 POST "/api/students" 201 7ms
10:50:48.565 GET "/api/students" 200 2ms
10:50:48.568 GET "/api/students/6ae47688dd1009b8dd63bf5c" 200 1ms
10:50:48.570 PATCH "/api/students/6ae47688dd1009b8dd63bf5c" 200 1ms
10:50:48.572 PUT "/api/students/6ae47688dd1009b8dd63bf5c" 200 1ms
10:50:48.573 DELETE "/api/students/6ae47688dd1009b8dd63bf5c" 204 1ms
10:50:48.574 GET "/api/students/6ae47688dd1009b8dd63bf5c" 404 1ms
HTTP CRUD: PASS
```

Рисунок 4 - Фрагмент сохранённого журнала MongoDB

Рисунки 3 и 4 подготовлены из фактических журналов, а не являются снимками Терминала. Общий набор из 56 тестов включает тестовое хранилище и настоящую выбранную БД; это один набор, повторённый для двух СУБД.

9 Сравнение PostgreSQL и MongoDB

Критерий	PostgreSQL	MongoDB
Структура хранения	Таблицы со строками, столбцами, типами и ограничениями.	Коллекции BSON-документов; возможны вложенные объекты и массивы.
Организация связей	Первичные и внешние ключи, объединение таблиц с JOIN.	Вложение данных либо ссылки по id; объединение через $lookup.
Удобство CRUD	SQL-команды SELECT, INSERT, UPDATE, DELETE; pg возвращает строки и число изменений.	Методы драйвера find, insertOne, updateOne, deleteOne и операции с возвратом документа.
Корректность данных	Типы, NOT NULL, UNIQUE, CHECK и внешние ключи.	JSON Schema и уникальные индексы; целостность ссылок обычно контролирует приложение.
Разработка и сопровождение	Удобна для устойчивой схемы и связанных сущностей; нужны миграции схемы.	Удобна для вложенных и меняющихся документов; требуется контроль схемы и дублирования.

Обе СУБД подошли для реализации студенческого CRUD. Одинаковые маршруты позволили проверить их на одном HTTP-контракте. Выбор модели хранения определяется структурой данных и требуемыми связями.

Для расширения системы таблицами групп, дисциплин и оценок PostgreSQL удобен благодаря ограничениям ссылочной целостности. MongoDB удобна, когда карточка студента содержит меняющиеся вложенные сведения. Это вывод для рассматриваемых сценариев, а не универсальное преимущество одной системы.

В текущей работе поле group_name является строкой. Отдельная сущность группы и связь с ней не реализованы.

10 Ответы на контрольные вопросы

1 Что такое Node.js и для каких задач он применяется?

Node.js - среда выполнения JavaScript вне браузера. Применяется для серверов, API, сетевых приложений, консольных утилит и работы с файлами. Неблокирующий ввод-вывод позволяет обслуживать множество ожидающих операций. Длительные вычисления в основном потоке могут ухудшать отзывчивость.

2 Какую роль выполняет Express.js?

Express - веб-фреймворк для Node.js. Он предоставляет маршруты, цепочки middleware и удобные средства обработки запросов и ответов: req.params, res.status(), res.json() и другие.

3 Что такое маршрутизация в Express?

Это сопоставление HTTP-метода и пути запроса с обработчиком. Например, router.get("/", handler), подключённый к /api/students, обрабатывает получение списка студентов.

4 Чем отличаются req.params, req.query и req.body?

req.params содержит параметры пути: для /api/students/7 значение id равно строке "7". req.query содержит параметры после ?, например ?group=ИС-21. req.body содержит разобранное тело запроса после express.json(). Все три источника требуют проверки; фильтрация через query в этом проекте не реализована.

5 Что такое middleware и для чего он используется?

Middleware - функция в цепочке обработки запроса. Она может разобрать JSON, записать журнал, проверить данные, установить заголовки, отправить ответ или передать управление следующему обработчику.

11 Продолжение контрольных вопросов

6 Как middleware передаёт управление следующему обработчику?

Через next(). Вызов next(error) передаёт ошибку обработчику ошибок. Если middleware не отправляет ответ и не вызывает next(), запрос может остаться незавершённым.

7 Каково назначение express.json()?

Это встроенный middleware для разбора JSON-тела запросов с подходящим Content-Type. Результат помещается в req.body. Он проверяет синтаксис JSON, а проверку обязательных полей и бизнес-правил выполняет код приложения.

8 Что такое API и REST API?

API - интерфейс взаимодействия программ. REST - архитектурный стиль, организующий взаимодействие через ресурсы и единообразный интерфейс. В HTTP API ресурсы обозначаются URI, операции выражаются методами, а результаты - статусами и представлениями ресурсов. Каждый запрос должен содержать необходимый контекст обработки. JSON распространён, но REST не требует именно JSON.

9 Что означает CRUD?

Create - создание, Read - чтение, Update - обновление, Delete - удаление. В проекте им соответствуют POST, GET, PUT или PATCH и DELETE.

10 Чем отличаются POST, PUT и PATCH?

В этом API POST создаёт нового студента, PUT передаёт полное состояние существующего студента, PATCH изменяет отдельные поля. PUT идемпотентен: повтор одинакового запроса приводит к тому же состоянию. POST обычно неидемпотентен. PATCH по стандарту не обязан быть идемпотентным, хотя присваивание значений полям в данной реализации идемпотентно.

12 Итог работы и состав архива

В результате разработан REST API управления студентами с шестью маршрутами и обработкой ошибок. Реализованы middleware логирования, проверка входных данных и постоянное хранение в PostgreSQL либо MongoDB. Оба варианта прошли автоматические и реальные HTTP-проверки.

Работа закрепила создание приложения Express, разделение маршрутов и слоя доступа к данным, использование HTTP-статусов, выполнение CRUD и применение ограничений целостности в двух СУБД.

Воспроизведение результата

```
npm install
npm run db:init
npm run db:seed
npm start

# Проверка соответствующего хранилища
npm run verify:postgres
npm run verify:mongodb
```

Перед проверкой необходимо подготовить рабочую и отдельную тестовую БД. На данном Mac они уже созданы. Основной режим в локальном .env - PostgreSQL; для MongoDB устанавливается DB_DRIVER=mongodb. Успешные журналы находятся в evidence/.

Архив содержит весь исходный код, package-lock.json, .env.example, SQL-схему, скрипты подготовки и проверки, тесты, README.md, журналы и этот отчёт в DOCX и PDF с изображениями. Локальный .env с паролем и автоматически устанавливаемый node_modules исключены.

Источники

Express - маршрутизация
https://expressjs.com/en/guide/routing/

node-postgres - параметризованные запросы
https://node-postgres.com/features/queries

PostgreSQL - ограничения данных
https://www.postgresql.org/docs/16/ddl-constraints.html

MongoDB - проверка схемы
https://www.mongodb.com/docs/manual/core/schema-validation/

MongoDB Node.js - операции с возвратом документа
https://www.mongodb.com/docs/drivers/node/current/crud/compound-operations/